

Personal Smart Locker Project

Fall 2026

Small, specialized circuits are the hearts of many modern electronic devices. These circuits cannot typically be programmed for general use but instead perform only one function very consistently and accurately. Whether you're designing a circuit for a washing machine, thermostat, smart watch, or television remote, your goals include being accurate, efficient, inexpensive, and easy to use. Most of these devices need to take actions based on different inputs (like which button was pressed or the value of a sensor), so they need to consider some combination of sensor values, past actions, and user input to decide what their next action is.

One example of a commercial product is personal smart lockers, which are used for temporary package storage. Whether your device is intended for a single home (see Figure 1), or for a small community (see Figure 2), these lockers allow packages to be delivered in a safe, secure manner without the risk of damage or theft of your package. The overall design goals of a system that can lock and unlock using a password can be applied to many scenarios (such as electronic door locks, garage door openers, and digital safes), for this project you'll be designing and implementing the electronics needed for a personal smart locker.



Figure 1: Example of a Personal Smart Locker



Figure 2: Example of a Commercial Smart Locker used near Virginia Tech

Project Description

The main focus of this project is developing the electronics and control for a Personal Smart Locker. This design should include the base functionality of being able to lock and unlock but will also include additional features that enhance the user experience and make it a complete system that meets user needs and competes with other devices on the market. These additional features include aspects that users look for in a product, such as ease of use and interaction,

customizability with changing password(s), a good battery life, and the ability to prevent damage to the mechanical components. With all of these features, it is especially important to have a robust method for choosing the correct action based on the current inputs and state, which can be accomplished using a state machine.

State Machines

A state machine is a way to model (and implement) the behavior of a complex system. Rather than continuously monitoring various inputs and checking multiple conditions, a state machine allows you to define how your system moves between states (or stays in its current state). By defining the states and transitions thoughtfully, you can ensure that your system never unintentionally leaves its state or enters the wrong state.

Consider the state transition diagram for a Smart Locker in Figure 3 below, where we want a system that can lock, unlock, and prevent the motor from running if it is blocked. In this state machine, let's assume the system begins in the locked state. If the correct password is entered, it transitions to the unlocked state, but if the password is incorrect, it remains in the locked state. In the unlocked state, if the lock button is pressed, it will move to the locking state, otherwise it will stay in the unlocked state. If it is in the locking state and detects something blocking the motor, the motor will move back to the unlocked state, otherwise it will move to the locked state.

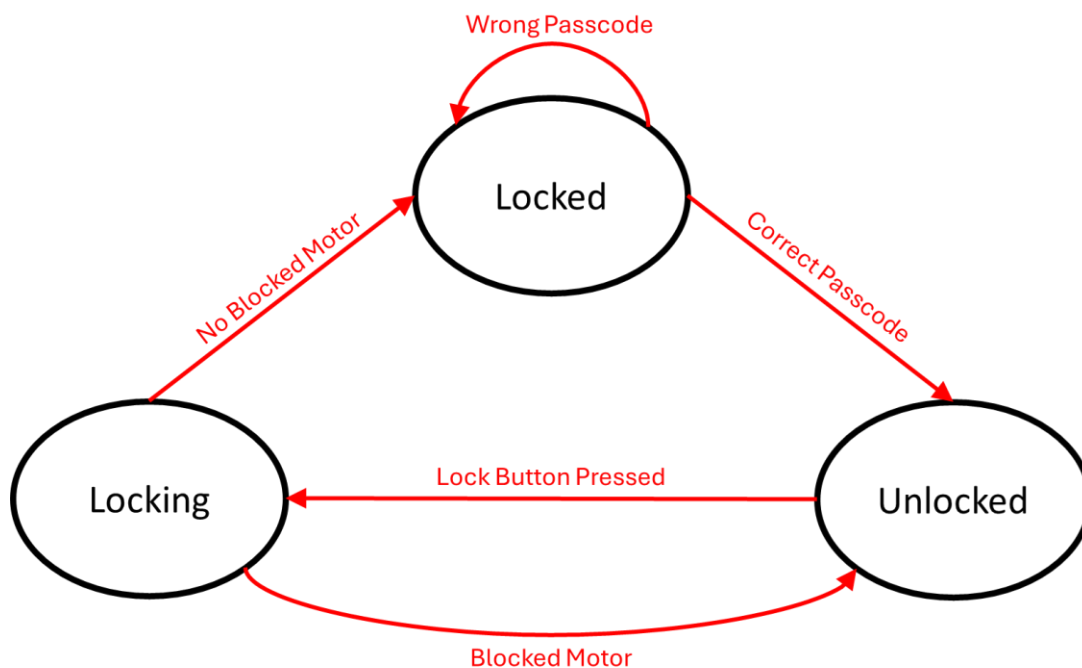


Figure 3: State Transition Diagram for a Smart Locker

If this state machine is implemented properly, the system can only change states on the defined transitions. Without a state machine, your design needs to be carefully programmed not to accidentally skip any steps. Otherwise, your system could move to the wrong step, which would cause the device to not work properly and may result in physical damage, such as if the code skips the step of monitoring the motor for being blocked. While this example seems easy, the system you will build for this project will get much more complicated with more inputs, features, and transitions to consider.

12-Key Matrix Keypad

Often hardware used for inputs (like keyboards) have all the hardware and software included that you need to get input from the system, requiring only the automatic download of a driver to start using the hardware. To give you experience in hardware and software design during this project, you'll receive the specialized hardware needed and will be tasked with designing your own hardware/software system to handle inputs from the keypad.

For this project, you will receive a 12-key matrix keypad for entering passwords and navigating between states of your system, whose wiring diagram can be found in Figure 4. Instead of using a separate input/output for every button, this keypad uses connections shared between multiple buttons. As you can see in this figure, each row and column share a connection to all buttons in that row column, but no two buttons pressed will connect the same row and column. This setup allows you to uniquely identify which button is pressed at any time using fewer pins to your microcontroller. However, using 7 pins may still be cumbersome in your hardware or software design, so consider ways to use this keypad that would allow you to use fewer pins of your microcontroller.

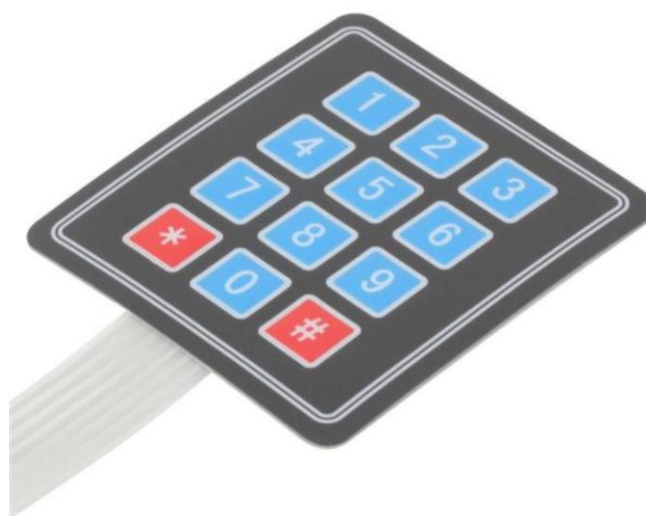


Figure 4: Example of 12-Key Matrix Keypad buttons

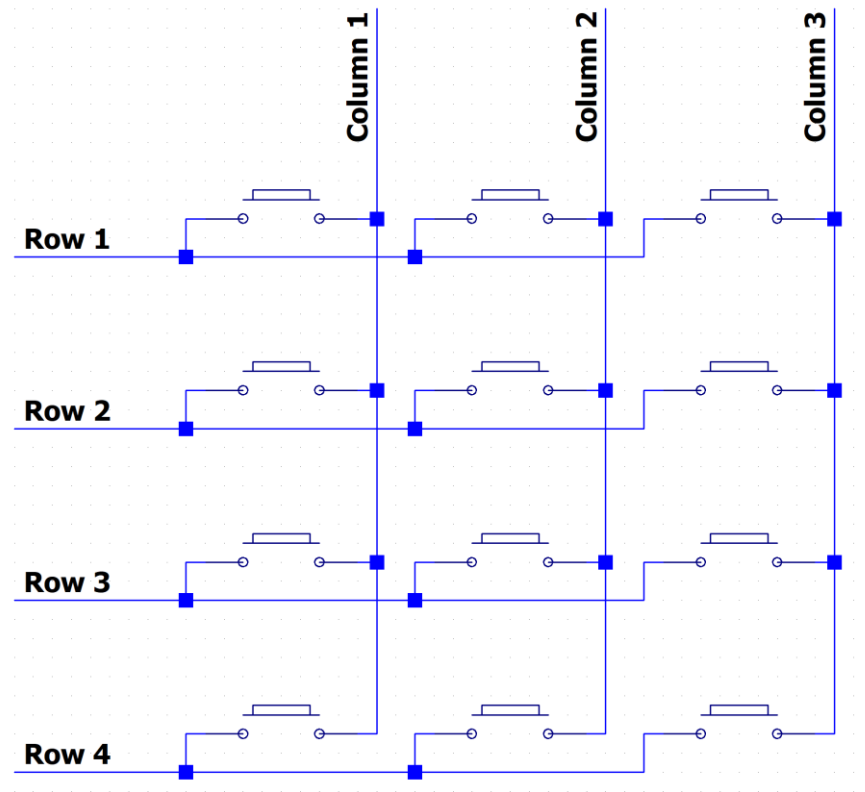


Figure 5: 12-Key Matrix Keypad Internal Connections

Servo Motor Control

Servo motors are used when you want to accurately and precisely adjust the angle of a motor shaft to specific angles. By using a potentiometer to measure the angle of the servo and then comparing it to the desired angle, this difference (the error) is used by an onboard control circuit to adjust the current the motor receives and move this error near zero, thus moving the servo to the desired angle and holding it there. This method is convenient for accurate angle placement and for holding the servo at that angle, as any additional forces on the motor that try to move in will increase the error term and the control circuit will attempt to account for it. However, the use of the potentiometer prevents servo motors from being able to rotate continuously.

When using servo motors with an onboard control circuit, the user only needs to provide the desired angle, and the control circuit will take care of rotating the motor to the position and holding it at the desired angle. For servo motors, this desired angle comes from a pulse width modulation (PWM) signal. While most applications that use PWM rely on the duty cycle of the signal (the percentage of a period where the signal is high), servo motors are controlled by the pulse width of the signal (the amount of time in which the signal is high). For the servo motors

used in this project, a pulse width of **roughly 1500 μ s corresponds to an angle of about 90°**, with a slightly larger or smaller pulse width changing the angle accordingly from 90°. Each motor is slightly different due to component tolerance, so you will need to test and determine what pulse width corresponds to what angle. The commonly used frequency for these servos is **50Hz**, so you will need to be able to generate a 50Hz PWM signal with a variable pulse width to control the motor for this project. Figure 5 below shows an example of varying pulse widths and servo motor angles. Note that the values in the figure are not necessarily the same values for your servo motor.

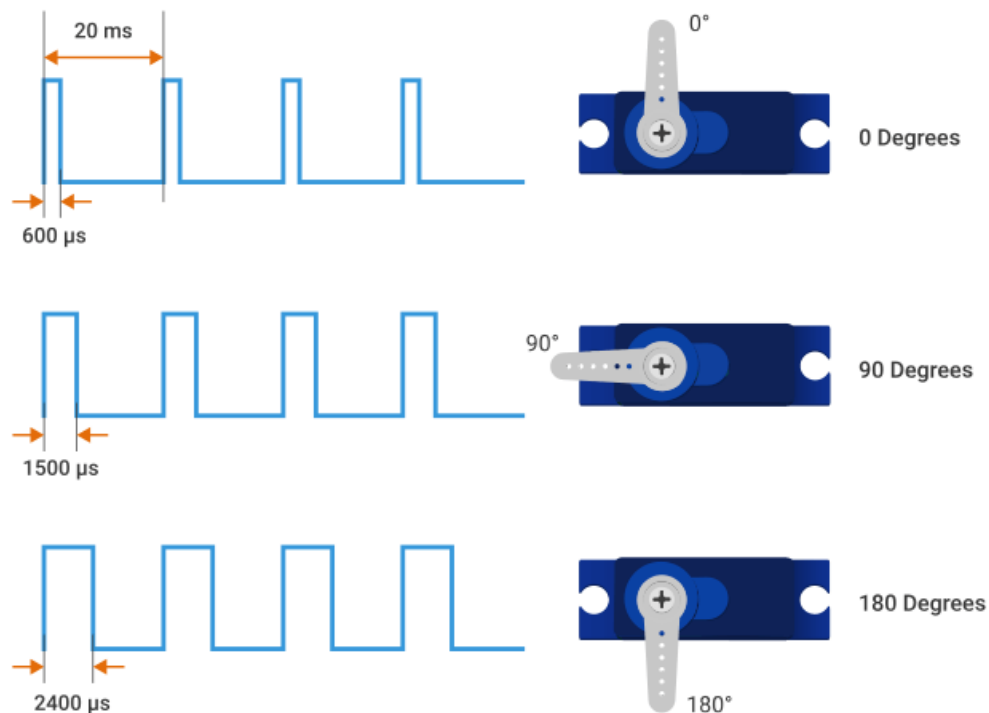


Figure 6: Example showing relationship between pulse width and servo motor angle

Notes:

1. To encourage the development and understanding of your own software systems, you cannot use any external libraries for the Arduino (nothing that requires a `#include` statement other than `math.h`) for any of the **base** project requirements. If your group adds anything additional, you're welcome to use other libraries.
2. You can use any components and parts from your parts kit; for this project, you will be provided with the following parts:

Parts List:

- SG90 micro servo motor
- 12-key matrix keypad
- Passive piezoelectric buzzer
- ATmega328p microcontroller
- MCP1702 5V voltage regulator
- 2x 1uF capacitors
- 16MHz crystal oscillator
- 2x 22pF capacitors
- 9V battery

Milestones and Deliverables

Milestone 1:

This milestone focuses on background research needed to design various subsystems for your overall project.

- a) While control of servo motors is easy if you use external Arduino libraries, it is important to understand how they work and how to implement them yourselves. For this project, you'll need to implement your own control of a servo motor without using external Arduino libraries. Use resources online to learn about how servos are controlled and write a short explanation along with a short pseudocode demonstrating how you'll turn on your servo, control it to a specific angle, and turn it off as needed. Note that you'll need to monitor and control the rest of your system at the same time, so this code should not be blocking.
- b) For this project, you'll use a standard 12-key number pad to get input from the user. It would be very inefficient and expensive for these keypads to have a set of wires for each button, so instead each column and row has its own connection, requiring only 7 total connections to your system to use. However, this number of connections may still be more than you have available. Make a plan to create a resistor bridge to determine the button pressed using only a single Analog Input pin, and justify why you chose those specific values.
- c) For your final design to be as cost and power efficient as possible, you will need to use the ATmega328P microcontroller chip rather than the Arduino Uno to save power and reduce cost of your design. While parts used to take this chip "offboard" are standard, you will have more options with how you enter and exit power saving mode to reduce the power

used by the system when it is idle. Using the Atmega328P datasheet, explore different power saving modes and methods to enter and exit them and write up your intended mode and how you plan to implement it, justifying why you chose that mode and how it will work.

- d) One of the more complex aspects of this project will be writing a program to control and monitor the entire system at once. For this scale of project, state machines can be used to ensure your states and transitions between states are completely defined and you can minimize the chance of jumping between states unexpectedly, as this may lead to bugs in your system. To get started on thinking about the logic behind your program, draw a detailed state transition diagram showing the different states of your system and how it will move in between states. This will ultimately become more complex as you add functionality to your system, but for now, consider a simplified system where you need to be able to:

- Power on the device when a button is pressed
- Enter a sleep mode when a certain amount of time has elapsed
- Receive input from a user for the password
- Check the password against the saved value
- Unlock the smart locker on a correct password
- Lock the smart locker when the lock button is pressed
- Display an error message/tone/etc. on an incorrect password

Additionally, write out a pseudocode that demonstrates how you could check your inputs and decide what state to transition to in this state machine.

Milestone 2:

This milestone focuses on getting the basic system working, including the keypad, servo motor control, and state machine of a basic locking system.

- a) Create a basic system with the keypad and your Arduino Uno that can tell when a key has been pressed and differentiate between the keys. Remember you can only use one AnalogInput pin (and no additional Digital pins). For this task, you need to demonstrate what key is pressed through some graphical means, such as using a series of LEDs or printing the key number to the Serial Terminal. (25% of milestone)
- b) Write a program that can manually control the angle of the servo motor without any external libraries. Make sure you have a plan for driving the servo motor, as the pins on the Arduino Uno (and ultimately the ATmega328P) cannot drive enough current itself. For this task, you need to demonstrate turning the motor to two positions (locked and unlocked). Make sure to look at specific values in the datasheet for the servo motor used in this project. (25%)

- c) Using your work on this Milestone and your state machine diagram you developed in Milestone 1, create a functioning system that can input a password, check the password based on a saved value, unlock the system if the password is correct, and relock the system when the lock button is pressed. (25%)
- d) Determine and implement a way to save a password to long-term memory in the Arduino so that the password is not hardcoded and will be persistent even if the system is completely disconnected from a power source. Show that this password works to unlock the system after losing power without having it hardcoded. Remember that you cannot use external libraries. (25%)

Milestone 3:

This milestone focuses on getting the system working without the Arduino Uno by using an ATmega328p chip that runs off a battery and minimizes power use.

- a) You'll need to monitor the servo motor to make sure it isn't blocked when trying to rotate (as this will draw excess current which will drain the battery and could damage the motor or microcontroller). Develop a way to determine if the servo motor is blocked and implement it so that the system reverts to its previous state to prevent damage. Your system should respond to a blocked motor quickly, within 1 second. Make sure you can tell the difference between a blocked servo motor and a servo motor that is moving to a new position. **Note:** When working with the servo, do not put stress on the motor for longer than a second or try to manually turn the servo when it is holding a position. If your blockage detection does not work immediately, stop putting stress on the motor or you may strip the gears and be required to purchase a new motor. (30% of milestone)
- b) Instead of using the Arduino Uno, adjust your circuit to run the same program from Milestone 2c off the ATmega328p instead. The whole system should now run off the provided 9V battery, so you'll need to determine how to provide the ATmega328P with a constant 5V. To demonstrate that you completed this task, you should show the new system can perform the exact same actions as Milestone 2c. Check out the resources at the end of this Project Description for a guide on getting code onto the ATmega328p. (40%)
- c) To save additional power, your system should not be running at all times, so you'll need to implement a method using hardware and/or software to minimize power usage. For this task, your goal is to minimize power consumption to make the system last as long as possible. Develop and implement a plan to minimize idle current while ensuring your

system can still become active when needed, and demonstrate that the system works with all functionalities you've developed so far (entering a code, locking and unlocking by controlling the servo, using the ATmega328p, etc.). (30%)

- a. For a hardware-focused solution, consider how you could cut power to parts of the system or even the whole system and monitor for user input to restore power to the system. With this type of implementation, there may be small amounts of current flowing through components (like the servo) even if you are not using it, so consider how to cutoff (ideally all) power until the user wants to use the system again.
- b. For a software-focused solution, consider how to implement a sleep mode for your microcontroller. These sleep modes are built into your microcontroller and can allow you to reduce current by turning off functions of the microcontroller that are not needed. The more you can turn off, the more power you'll save, but make sure you can exit the sleep mode when desired.

Milestone 4:

This milestone focuses on functions that improve the User Interface and Security of the system. It shouldn't just work as a locker but have additional features that draw users to choose your implementation over other options! **Note:** It is really easy to drain your battery if you leave the system plugged in while testing (depending on the current draw of your system), so consider **removing** the 5V regulator and replacing it with the 5V source of your AD2 whenever you are not testing the battery or if you are making changes to your code or system that may unintentionally increase the current draw and drain your battery. Make sure to switch back to the battery for any Milestone or Verification/Validation testing though.

- a) Your system should monitor and alert the user if the battery is getting low so they can replace it before it dies (which could make it much harder to access to replace). Implement a method for checking the battery status and alerting the user of a low battery. Justify how you can determine if the battery is low. Make sure to not constantly draw significant current when checking and alerting the user of the low battery, as you do not want to significantly reduce the battery life just by checking it. (15% of milestone)
- b) Implement **both** of the following "add-on" User Interface options for your system: (15%)
 - LEDs that show the state of the system (enter password, wrong password, reprogramming, etc.) to the user to make the system easier to operate
 - A passive piezoelectric buzzer to indicate states to the user (correct password, wrong password, etc.)

- c) Implement a method for securely saving the password so if anyone gains access to the physical device and its memory, they will not be able to determine the password. This method should be secure enough such that if they gained access to your device's memory, they still could not determine the password (i.e., you cannot use any simple methods that are easily reversible). You may consider looking into hashing algorithms, but you are also welcome to design, implement, and justify your own method for storing a protected (non-plaintext) password. Explain what method you implemented to securely store the password, **how it works**, and **why this method is secure for** this purpose. Remember, you cannot use external libraries to implement this. (30%)
- d) Your system should last as long as possible before the user has to replace the battery. With all of the User Interface and Security options implemented, measure the mAh used by the system when it is sleeping (not checking for passwords) and the mAh used by the system when it is active (checking for passwords and turning the motor twice, once to unlock and once to lock). Using these values, calculate the number of days the system can be unactive for (assuming a 600mAh 9V battery) and the number of times it can wake up and lock/unlock the system on a full battery. Discuss what changes you can make to your system (while keeping the system reasonably user-friendly) to reduce the mAh used in each of these scenarios, and implement some of your ideas showing how much you are able to extend the battery life. Your system should last **at least** 6 months if it is not active, but your goal is for the system to run off a single battery for the entire shelf life of the battery (about 5 years) assuming reasonable usage of the smart locker while keeping it reasonably user-friendly. (30%)
- e) Demonstrate that the entire system works with all required features and the additional optional features that your team selected. (10%)

Design Verification:

In your final report, you must verify that your design meets the following specifications:

1. Show that your program can successfully determine all of the buttons on the keypad without any errors
2. Show that PWM pulse width measurements for the open and close states of your servo match the intended pulse widths
3. Show that when the servo is blocked from turning, the system reverts back to the previous state to prevent overcurrent within 1 second, along with measurements detailing how your system determined the motor was blocked

4. Show that your system can successfully enter and exit sleep mode when idling for too long without user input
5. Calculate (with evidence) how long your final system can last in sleep mode, and how many total times it could go through an awake/lock/unlock cycle on a full battery
6. Show that the password can be changed and will remain stable even when the system is fully disconnected from power
7. Show and explain how your User Interface option helps communicate to the user what state they are in

Design Validation

Include a design validation video showing the entire system working at the same time. This video should demonstrate that the system can enter and exit sleep mode appropriately while running on a 9V battery, unlock the system on a successful password, lock the system when the lock button is pressed, revert to the previous state if the motor is blocked, change the password and have it persist through the system being disconnected from power, and the additional add-on features from Milestone 4. To help illustrate that the system is working correctly, you should start the video showing and discussing an **updated** state transition diagram of the completed project that shows what actions occur between states (e.g., motor turns, LEDs light up, buzzer sounds, etc.).

Resources

1. SG90 Servo Motor datasheet on Canvas
2. ATmega328p microcontroller datasheet guide on Canvas
3. ATmega328p offboard programming guide on Canvas
4. MCP1702 LDO Regulator datasheet on Canvas
5. Watch the following short videos from our friends at MATLAB/Mathworks to review some basic concepts regarding state machines:
 - a. Understanding State Machines, Part 1: What Are They? - [State Machines, Part 1](#)
 - b. Understanding State Machines, Part 2: Why Use Them? - [State Machines, Part 2](#)
 - c. Understanding State Machines, Part 3: Mealy and Moore Machines - [State Machines, Part 3](#)
 - d. Understanding State Machines, Part 4: Harel State Machines – [State Machines, Part 4](#)